

Оглавление

Оглавление.....	1
1 Введение	2
2 Современные архитектурные стили WEB-приложений.....	3
2.1 Монолитная архитектура.....	3
2.2 Многоуровневая (N-tier) архитектура	4
2.3 Микросервисная архитектура	5
2.4 Микро-фронтенды	6
2.5 Serverless (бессерверная) архитектура.....	7
2.6 Клиентские паттерны: SPA, MPA, PWA.....	8
3 Платформа Delphi 10.2 Токуо для разработки WEB-приложений	9
3.1 Основные технологии и фреймворки для WEB-разработки	10
3.2 Описание ключевых технологий.....	11
3.3 Интеграция и архитектурные подходы	12
3.4 Оценка возможностей Delphi 10.2 как платформы для WEB-разработки.....	12
3.5 Варианты использования в компании	13
3.6 Вывод по Delphi 10.2	14
4 Архитектура платформы «1С: Предприятие» для WEB-приложений.....	14
4.1 Классическая трёхуровневая архитектура.....	14
4.2 Веб-клиент и PWA	15
4.3 Интеграция и SOA	15
4.4 1С в контексте современных архитектур.....	15
4.5 Оценка возможностей 1С как WEB-платформы	16
4.6 Варианты использования 1С в компании	16
5 Microsoft Internet Information Services (IIS) как платформа для WEB-приложений	17
5.1 Архитектура IIS.....	17
5.2 Сравнение с другими веб-серверами	18
5.3 Варианты использования IIS в компании	19
5.3.1 Вывод по IIS	19
6 Сравнительный анализ архитектурных подходов.....	20
7 Обзор популярных фреймворков и платформ для WEB-разработки.....	21
8 Варианты использования архитектур в компании	22

8.1	Стартапы и малые компании	22
8.2	Средние компании (50–200 сотрудников).....	22
8.3	Крупные компании и высоконагруженные системы	22
8.4	Компании с переменной нагрузкой.....	23
8.5	Компании с несколькими фронтенд-командами	23
8.6	Компании, работающие в российской юрисдикции с учётными задачами	23
9	Рекомендации по выбору архитектурного решения	23
10	Заключение	24
11	Список использованных источников.....	27

1 Введение

В современном цифровом ландшафте архитектура WEB-приложений является одним из ключевых факторов, определяющих успех цифровых продуктов. От правильного выбора архитектурного решения зависят масштабируемость, производительность, стоимость разработки и сопровождения, а также способность системы адаптироваться к изменяющимся требованиям бизнеса.

На сегодняшний день на рынке программного обеспечения представлено множество архитектурных подходов — от классических монолитных систем до современных микросервисных и бессерверных решений. Информация об этих архитектурах часто разрознена, что затрудняет выбор оптимального решения. Цель настоящего анализа — систематизировать знания о существующих архитектурах WEB-приложений, оценить их возможности и предложить рекомендации по выбору в зависимости от потребностей компании.

В работе рассматриваются основные архитектурные стили, наиболее распространённые в современной веб-разработке, а также анализируется платформа «1С:Предприятие» как важный игрок на российском рынке корпоративных информационных систем.

2 Современные архитектурные стили WEB-приложений

2.1 Монолитная архитектура

Монолитная архитектура представляет собой единое приложение, в котором все компоненты — пользовательский интерфейс, бизнес-логика и работа с данными — объединены в один исполняемый модуль.

Преимущества:

Простота разработки и развёртывания — монолитные системы проще всего настраивать и управлять ими.

Единый технологический стек — все команды используют одинаковые инструменты и языки программирования.

Упрощённое тестирование — легче выполнять сквозное тестирование, так как все компоненты находятся в одном процессе.

Меньше накладных расходов — нет необходимости в сложной сетевой коммуникации между сервисами.

Недостатки:

Ограниченная масштабируемость — масштабируется только всё приложение целиком, даже если нагрузка приходится только на один модуль.

Сложность поддержки — по мере роста кодовой базы монолит становится трудным для понимания и модификации.

Технологическая связанность — сложно внедрять новые технологии, так как они должны поддерживаться всем приложением.

Риск при развёртывании — даже небольшое изменение требует полной перекомпиляции и переразвёртывания всего приложения.

Когда применять:

Небольшие проекты и стартапы на начальных этапах.

Внутренние корпоративные системы с ограниченным функционалом.

Проекты с небольшим количеством разработчиков (до 10 человек).

2.2 Многоуровневая (N-tier) архитектура

Многоуровневая архитектура разделяет приложение на логические уровни, каждый из которых выполняет определённые функции. Классическая трёхуровневая архитектура включает уровень представления (frontend), уровень бизнес-логики (backend) и уровень данных (база данных).

Преимущества:

Чёткое разделение ответственности — каждый уровень можно разрабатывать и сопровождать независимо.

Умеренная масштабируемость — каждый уровень можно масштабировать отдельно.

Гибкость в выборе технологий — разные уровни могут использовать различные технологии.

Улучшенная безопасность — можно размещать уровни в разных сетевых сегментах.

Недостатки:

Сложнее монолита — требуется больше усилий на настройку и управление.

Сетевые задержки — коммуникация между уровнями через сеть может увеличивать время задержки.

Сложность транзакций — распределённые транзакции сложнее реализовать и поддерживать.

Когда применять:

Классические корпоративные приложения (ERP, CRM).

Проекты среднего размера с чётким разделением на frontend и backend.

Системы, где важна безопасность и разграничение доступа.

2.3 Микросервисная архитектура

Микросервисная архитектура разбивает приложение на множество небольших, независимо развёртываемых сервисов, каждый из которых отвечает за одну бизнес-функцию. Сервисы взаимодействуют друг с другом через API и могут использовать различные технологии и базы данных.

Преимущества:

Независимое масштабирование — можно масштабировать только те сервисы, которые испытывают высокую нагрузку.

Независимое развёртывание — каждый сервис можно обновлять без остановки всего приложения.

Технологическая гибкость — каждый сервис может использовать свой технологический стек.

Устойчивость к сбоям — отказ одного сервиса не приводит к падению всей системы.

Удобство для больших команд — разные команды могут разрабатывать разные сервисы параллельно.

Недостатки:

Высокая сложность — микросервисные системы значительно сложнее в разработке и управлении.

Сетевые задержки — коммуникация между сервисами через сеть вносит дополнительную задержку.

Сложность отладки — трассировка запросов через множество сервисов требует специальных инструментов.

Управление данными — распределённые транзакции и согласованность данных становятся нетривиальными задачами.

Требует зрелой команды — необходим высокий уровень компетенций в DevOps и распределённых системах.

Когда применять:

Крупные распределённые системы с высокой нагрузкой (Netflix, Amazon).

Проекты с несколькими независимыми командами разработки.

Системы, требующие частых обновлений и быстрого вывода новых функций.

Компании, готовые инвестировать в инфраструктуру и DevOps-культуру.

Важно: Согласно исследованиям 2025 года, микросервисная архитектура демонстрирует значительно более высокую устойчивость к сбоям по сравнению с монолитом — до 12% снижение частоты отказов при моделируемых сбоях. Однако монолит может превосходить микросервисы по пропускной способности и задержкам при большинстве сценариев нагрузки.

2.4 Микро-фронтенды

Микро-фронтенды — это подход, при котором пользовательский интерфейс разбивается на независимые модули, каждый из которых разрабатывается и развёртывается отдельной командой.

Преимущества:

Параллельная разработка — разные команды могут работать над разными частями интерфейса независимо.

Независимое развёртывание — можно обновлять отдельные части интерфейса без перезагрузки всего приложения.

Технологическая гибкость — разные модули могут использовать разные фреймворки.

Недостатки:

Сложность интеграции — требуется координация между командами для обеспечения целостного пользовательского опыта.

Увеличение размера бандла — загрузка нескольких независимых модулей может увеличить время загрузки страницы.

Когда применять:

Крупные проекты с несколькими командами фронтенд-разработчиков.

Системы, где разные части интерфейса обновляются с разной частотой.

Миграция с монолитного фронтенда на более гибкую архитектуру.

2.5 Serverless (бессерверная) архитектура

Serverless — это парадигма вычислений, при которой разработчики создают и запускают программное обеспечение без необходимости управления инфраструктурой. Провайдер (AWS Lambda, Azure Functions, Google Cloud Functions) автоматически управляет серверами, масштабированием и обеспечением доступности.

Преимущества:

Отсутствие управления серверами — разработчики сосредотачиваются на коде, а не на инфраструктуре.

Автоматическое масштабирование — система автоматически адаптируется к нагрузке.

Экономическая эффективность — оплата только за фактическое время выполнения, а не за простаивающие серверы.

Быстрая разработка и развёртывание — функции обычно небольшие и модульные.

Недостатки:

Холодный старт — задержка при первом вызове функции после периода бездействия.

Привязка к вендору — миграция между провайдерами сложна.

Ограничения по времени выполнения — большинство платформ имеют лимиты на время выполнения функции (например, AWS Lambda около 15 минут).

Сложность отладки — традиционные инструменты отладки плохо работают в распределённой serverless-среде.

Когда применять:

Приложения с переменной и непредсказуемой нагрузкой.

Событийно-ориентированные системы (обработка файлов, уведомления, IoT).

Быстрая разработка MVP и прототипов.

Проекты, где важно минимизировать операционные расходы.

2.6 Клиентские паттерны: SPA, MPA, PWA

Single-Page Application (SPA) — приложение, которое загружает одну HTML-страницу и динамически обновляет её содержимое по мере взаимодействия пользователя, без перезагрузки всей страницы.

Преимущества: плавный пользовательский интерфейс, снижение нагрузки на сервер, богатая интерактивность.

Недостатки: проблемы с SEO, медленная первая загрузка, сложность управления состоянием.

Multi-Page Application (MPA) — традиционный подход, при котором каждое действие пользователя приводит к загрузке новой страницы с сервера.

Преимущества: лучше для SEO, проще в разработке, надёжнее работает в условиях плохого интернета.

Недостатки: более медленный пользовательский опыт, большая нагрузка на сервер.

Progressive Web App (PWA) — веб-приложение, которое использует современные веб-технологии для обеспечения возможностей, аналогичных нативным приложениям (офлайн-режим, push-уведомления, установка на устройство).

Когда применять:

SPA — для сложных интерактивных интерфейсов, дашбордов, админ-панелей.

MPA — для контентных сайтов, интернет-магазинов, где важен SEO.

PWA — для мобильных-ориентированных веб-приложений, где требуется офлайн-доступ.

3 Платформа Delphi 10.2 Tokyo для разработки WEB-приложений

Delphi 10.2 Tokyo (выпущенная в 2017 году) представляет собой мощную среду разработки, которая, вопреки распространённому мнению, предлагает широкий набор инструментов для создания WEB-приложений. В отличие от многих современных WEB-фреймворков, Delphi позволяет создавать высокопроизводительные, нативные серверные приложения, используя знакомую среду и язык Object Pascal.

Ключевой особенностью версии 10.2 стало введение поддержки 64-разрядной платформы Linux для серверных приложений. Это позволило размещать разработанные на Delphi сервисы не только в экосистеме Windows, но и на более экономичных и распространённых Linux-серверах

3.1 Основные технологии и фреймворки для WEB-разработки

Платформа Delphi 10.2 предлагает несколько подходов к созданию WEB-приложений, от встроенных в IDE решений до мощных сторонних фреймворков

Технология / Фреймворк	Тип / Архитектура	Ключевые особенности
WebBroker	Встроенная технология	Базовый каркас для создания веб-серверных приложений (ISAPI, CGI, WinCGI, Apache).
DataSnap	Встроенная технология	Создание многозвенных приложений и RESTful серверов с автоматической сериализацией данных.
RAD Server (EMS)	Встроенная платформа	Высокоуровневая платформа для быстрой публикации бизнес-логики в виде REST API.
IntraWeb	Встроенный фреймворк	Позволяет создавать веб-приложения визуально, как обычные VCL/FMX приложения.
uniGUI	Сторонний фреймворк	Разработка веб-приложений с помощью визуальных компонентов, мощнее IntraWeb.
DelphiMVCFramework	Сторонний фреймворк	Популярный open-source фреймворк для создания RESTful и JSON-RPC серверов.
DeWeb	Сторонний инструмент	Позволяет конвертировать существующие Delphi-приложения в веб-приложения.

3.2 Описание ключевых технологий

- WebBroker – это фундаментальная технология, предоставляющая основу для всех серверных веб-решений. Она интегрируется с веб-серверами через ISAPI (для IIS) или CGI и обрабатывает HTTP-запросы, направляя их на соответствующие действия (actions). WebBroker доступен для сборки как в виде ISAPI-библиотеки (DLL), так и в виде самостоятельного серверного приложения (EXE).
- DataSnap – это более высокоуровневая технология для создания многозвенных приложений. В версии 10.2 она была интегрирована с TWebModule, что позволило легко создавать RESTful серверы, автоматически генерировать REST-маршруты и сериализовать сложные объекты (например, TObjectList<T> и TDataSet) в JSON.
- RAD Server (EMS - Enterprise Mobility Services) – это готовая платформа для развертывания REST API. Начиная с Delphi 10.2.2, лицензия на использование RAD Server для одного сайта стала доступна в редакциях Enterprise и Architect. RAD Server позволяет быстро превратить существующую бизнес-логику, написанную на Delphi, в масштабируемые и защищённые HTTP-эндпоинты.
- IntraWeb и uniGUI – эти фреймворки предлагают совершенно иной подход, максимально приближенный к разработке классических десктопных приложений. Разработчик работает с визуальными компонентами (кнопки, поля ввода) в привычном дизайнера форм, а фреймворк автоматически генерирует HTML, CSS и JavaScript для отображения в браузере. Это позволяет командам, не имеющим глубоких знаний веб-технологий, создавать полноценные веб-приложения.
- DelphiMVCFramework – это самый популярный open-source проект на Delphi для веб-разработки. Он предоставляет полноценную MVC (Model-View-Controller) архитектуру для создания RESTful и JSON-RPC API. Фреймворк полностью поддерживает Delphi 10.2 Tokyo и

отличается простотой использования, высокой производительностью и богатой документацией.

3.3 Интеграция и архитектурные подходы

Delphi-решения могут быть интегрированы в современный ИТ-ландшафт несколькими способами:

- Как монолитное веб-приложение: С использованием IntraWeb или uniGUI можно создать полноценное веб-приложение, которое будет работать как единое целое.
- Как бэкенд-сервис (REST API): Наиболее современный подход. С помощью RAD Server, DataSnap или DelphiMVCFramework вы создаёте бэкенд, который предоставляет REST API. Фронтенд может быть реализован на любом современном фреймворке (React, Angular, Vue.js) или в виде мобильного приложения. Это соответствует принципам многоуровневой (N-tier) архитектуры.
- Как микросервис: Благодаря малому размеру, высокой производительности и возможности компиляции под Linux, сервисы на Delphi отлично подходят для микросервисной архитектуры. Можно развернуть несколько независимых Delphi-сервисов, каждый из которых отвечает за свою бизнес-функцию.
- Как ISAPI-модуль: Традиционный способ развертывания в экосистеме Windows с использованием IIS.

3.4 Оценка возможностей Delphi 10.2 как платформы для WEB-разработки

Критерий	Оценка (по 5-балльной шкале)	Комментарий
Простота разработки	★★★★☆	Для серверной логики и API — очень высокая. Для веб-интерфейсов (IntraWeb/uniGUI) — интуитивно понятна для Delphi-разработчиков.
Производительность	★★★★★	Нативные приложения, скомпилированные в машинный код, обеспечивают максимальную производительность.

Поддержка современных архитектур	★★★★☆	Полная поддержка RESTful API, микросервисов, развертывания на Linux.
Интеграционные возможности	★★★★☆	Поддержка JSON, REST, SOAP (через веб-сервисы), HTTP-запросов, работа с любыми СУБД через FireDAC.
Сообщество и экосистема	★★★★☆	Сообщество меньше, чем у Python или JavaScript, но есть активное ядро и качественные сторонние фреймворки (например, DelphiMVCFramework).
Стоимость	★★☆☆☆	Delphi — платный продукт с высокой стоимостью лицензий. Однако эта стоимость может окупаться скоростью разработки и производительностью для нишевых задач.

3.5 Варианты использования в компании

Сценарий использования	Описание
Создание высоконагруженных бэкенд-сервисов	Разработка REST API для мобильных приложений или SPA-фронтендов. Высокая производительность Delphi позволяет обслуживать большое количество запросов.
Миграция устаревших десктопных приложений в веб	С помощью IntraWeb или uniGUI компании могут перевести свои старые проекты на Delphi в веб без полной переписывания бизнес-логики.
Разработка внутренних корпоративных систем	Создание систем управления, дашбордов и отчетов, где важна быстрая разработка и интеграция с существующей ИТ-инфраструктурой на Windows.
Создание микросервисов	Разработка небольших, быстрых и независимых сервисов для

	специфических задач (например, обработка сложных финансовых расчетов, генерация документов, интеграция с внешними системами).
--	---

3.6 Вывод по Delphi 10.2

Delphi 10.2 Tokyo — это не архаичный инструмент, а современная, высокопроизводительная платформа, которая успешно конкурирует с популярными веб-фреймворками в нише создания серверных приложений и REST API. Её ключевая особенность и сильная сторона — в производительности и возможности разработки с использованием привычного языка и среды, особенно для компаний с существующим кодом на Delphi. В то время как для создания сложных и интерактивных пользовательских интерфейсов обычно выбирают React или Angular, Delphi становится идеальным выбором для надёжного, эффективного и масштабируемого бэкенда.

4 Архитектура платформы «1С: Предприятие» для WEB-приложений

Платформа «1С: Предприятие» является доминирующим решением для автоматизации бухгалтерского и управленческого учёта в России и странах СНГ. Однако она представляет собой не просто приложение, а полноценную платформу для разработки, способную функционировать в WEB-среде.

4.1 Классическая трёхуровневая архитектура

Приложения на платформе «1С: Предприятие» могут быть развернуты в рамках трёхуровневой архитектуры (клиент — сервер приложений — база данных):

Клиентский уровень — пользователь взаимодействует через:

Толстый клиент (традиционное приложение)

Тонкий клиент (устанавливаемый, но основная логика на сервере)

Веб-клиент — работает в браузере без установки (поддерживается в версиях 8.3 и выше).

Уровень веб-сервера — необходим для веб-клиента и тонкого клиента через HTTP. Поддерживаются Microsoft IIS и Apache.

Уровень сервера приложений (кластер 1С) — выполняет бизнес-логику, генерирует отчёты, управляет объектами. Клиенты не имеют прямого доступа к базе данных.

Уровень СУБД — используются промышленные СУБД: Microsoft SQL Server, PostgreSQL, Oracle DB, IBM DB2.

4.2 Веб-клиент и PWA

Веб-клиент 1С является сложным JavaScript-фреймворком. Код клиентских модулей на встроенном языке 1С автоматически компилируется в JavaScript и выполняется в браузере. Для обмена данными с сервером используются технологии AJAX (XMLHttpRequest). В последних версиях платформы поддерживается концепция Progressive Web App (PWA), что позволяет использовать веб-клиент как приложение на устройстве и работать в офлайн-режиме.

4.3 Интеграция и SOA

Платформа поддерживает сервис-ориентированную архитектуру (SOA):

1С как поставщик веб-сервисов — публикация бизнес-логики для вызова из других систем.

1С как потребитель веб-сервисов — обращение к внешним сервисам через WS-ссылки.

Это позволяет встраивать 1С в гетерогенные ИТ-ландшафты и взаимодействовать с микросервисами.

4.4 1С в контексте современных архитектур

Хотя классическая архитектура 1С является трёхуровневой и часто воспринимается как монолит, платформа может использоваться в гибридных схемах:

1С выступает как ядро для учётных функций, а высоконагруженные или специализированные задачи выносятся во внешние микросервисы.

Взаимодействие осуществляется через REST API или веб-сервисы.

Полный отказ от 1С в пользу микросервисов для крупных предприятий сложен, поэтому чаще применяется эволюционный гибридный подход.

4.5 Оценка возможностей 1С как WEB-платформы

Критерий	Оценка (по 5-балльной шкале)	Комментарий
Простота разработки бизнес-логики	★★★★★	Встроенный язык и предметно-ориентированные объекты ускоряют разработку учётных систем.
Наличие готовых решений	★★★★★	Огромное количество типовых и отраслевых конфигураций.
Гибкость архитектуры	★★★★☆	Классическая трёхуровневая архитектура надёжна, но менее гибка, чем чистая микросервисная.
Интеграционные возможности	★★★★☆	Поддержка веб-сервисов, HTTP, XML, JSON.
Качество WEB-интерфейса (UX/UI)	★★★★☆	Веб-клиент функционален, но уступает современным SPA-фреймворкам по кастомизации и анимации.
Стоимость владения	★★★★☆	Высокая стоимость лицензий и сопровождения, но окупается скоростью разработки для учёта.
Производительность и отказоустойчивость	★★★★☆	Кластерная архитектура и поддержка промышленных СУБД обеспечивают высокую производительность.

4.6 Варианты использования 1С в компании

Сценарий	Описание
----------	----------

Базовое корпоративное приложение	Внедрение типовых конфигураций (Бухгалтерия, Управление торговлей, Зарплата) с доступом через веб-клиент.
Ядро в гетерогенной среде	1С выступает как «система-источник истины» для финансовых данных; остальные системы интегрируются через веб-сервисы.
Разработка собственных отраслевых решений	Использование платформы 1С и инструмента 1С:EDT для создания уникальных прикладных решений под специфику бизнеса.
Облачный сервис (SaaS)	Развертывание 1С в облаке с предоставлением доступа через веб-клиент, что снижает затраты на ИТ-инфраструктуру.

5 Microsoft Internet Information Services (IIS) как платформа для WEB-приложений

Microsoft Internet Information Services (IIS) — это проприетарный набор серверов для нескольких интернет-служб от компании Microsoft. До версии 5.1 он назывался Internet Information Server. Основным компонентом IIS является веб-сервер, который позволяет размещать в интернете сайты и веб-приложения. IIS распространяется в составе операционных систем семейства Windows NT и является стандартным решением для развёртывания веб-приложений в среде Windows.

5.1 Архитектура IIS

Начиная с версии 7.0, IIS имеет **полностью модульную архитектуру**, что стало значительным шагом вперёд по сравнению с предыдущими версиями. Эта архитектура предлагает три ключевых преимущества:

- **Компонентизация (Componentization):** Все функции веб-сервера управляются как автономные компоненты, которые можно легко добавлять, удалять и заменять. Это позволяет:
 - **Повысить безопасность**, уменьшая область атаки за счёт удаления неиспользуемых функций.
 - **Улучшить производительность** и снизить потребление памяти, удаляя неиспользуемый код.
 - **Создавать специализированные серверы**, оптимизированные под конкретные задачи, например, для кэширования или балансировки нагрузки.

- **Расширяемость (Extensibility):** Разработчики могут создавать собственные модули для расширения или замены существующих функций веб-сервера.
- **Интеграция с ASP.NET:** Модульная архитектура обеспечивает тесную и эффективную интеграцию с платформой ASP.NET, что является ключевым преимуществом для разработчиков на C# и Visual Basic.
Ключевые архитектурные компоненты IIS включают:
 - **Сайты (Sites):** Представляют собой отдельные веб-сайты, идентифицируемые по уникальной комбинации имени хоста, порта и IP-адреса.
 - **Приложения (Applications):** Являются подмножеством сайта, которое работает как независимая единица и может иметь собственные настройки и пул приложений.
 - **Виртуальные каталоги (Virtual Directories):** Позволяют включать в структуру сайта содержимое из физических папок, расположенных за пределами корневого каталога сайта.
 - **Пулы приложений (Application Pools):** Обеспечивают изоляцию веб-приложений друг от друга. Каждый пул работает в собственном процессе (w3wp.exe), что предотвращает влияние сбоя одного приложения на другие.
 - **Модули (Modules):** Обработывают конкретные задачи, такие как аутентификация, сжатие, URL-переписывание и другие

5.2 Сравнение с другими веб-серверами

Для понимания места IIS на рынке полезно сравнить его с основными конкурентами — Apache и Nginx:

	Microsoft IIS	Apache	Nginx
Платформа	Windows Server	Кроссплатформенный	Кроссплатформенный
Архитектура	Модульная, основанная на потоках	Модульная, основанная на процессах (MPM)	Асинхронная, событийно-ориентированная
Производительность при высоких нагрузках	Средняя	Средняя	Высокая
Интеграция с экосистемой Microsoft	Отличная	Ограниченная	Ограниченная
Управление	Графический интерфейс (IIS Manager)	Конфигурационные файлы	Конфигурационные файлы

Лицензирование	Проприетарное (входит в состав Windows)	Открытое ПО (Apache License)	Открытое ПО (BSD-like)
Доля рынка	~3.5% (декабрь 2025)	Высокая	Высокая (лидирующая)

Ключевые выводы из сравнения:

- **Nginx** является лидером по производительности при высоких нагрузках и часто используется как обратный прокси.
- **Apache** остаётся надёжным и гибким решением с огромным сообществом и множеством модулей.
- **IIS** — это выбор по умолчанию для организаций, которые уже используют Windows Server и экосистему Microsoft (.NET, SQL Server, Active Directory). Он предлагает бесшовную интеграцию, удобное графическое управление и надёжную поддержку со стороны Microsoft.

5.3 Варианты использования IIS в компании

IIS может быть использован в компании в следующих сценариях:

- **Корпоративные веб-приложения и интранет-порталы:** Размещение внутренних систем, созданных на платформе .NET Framework или ASP.NET Core.
- **Хостинг веб-сайтов:** Размещение корпоративных сайтов, интернет-магазинов и других публичных веб-ресурсов в среде Windows.
- **Платформа для API:** Создание и хостинг RESTful API для мобильных приложений и внешних систем.
- **Интеграция с Active Directory:** Использование встроенной аутентификации Windows для обеспечения единого входа (SSO) в корпоративных сетях.
- **Среда для разработки и тестирования:** Благодаря лёгкости настройки на клиентских версиях Windows, IIS часто используется разработчиками для локального тестирования веб-приложений.

5.3.1 Вывод по IIS

Microsoft IIS — это надёжный и функциональный веб-сервер, который является неотъемлемой частью экосистемы Windows. Его ключевая сила

заключается в глубокой интеграции с продуктами и технологиями Microsoft, что делает его оптимальным выбором для организаций, чья ИТ-инфраструктура построена на базе Windows Server, .NET и SQL Server. Несмотря на меньшую долю рынка по сравнению с Nginx и Apache, IIS остаётся важным инструментом в корпоративном сегменте, предлагая высокий уровень безопасности, управляемости и производительности для специфических задач.

6 Сравнительный анализ архитектурных подходов

В таблице ниже приведена сравнительная оценка ключевых характеристик рассмотренных архитектур (по 5-балльной шкале, где 1 — наименьшее значение, 5 — наибольшее).

Сравнение архитектурных подходов

Критерий	Монолит	N-tier	Микросервисы	Serverless	Микро-фронтенды	1С (классическая)
Простота разработки	5	4	2	4	3	5
Масштабируемость	2	3	5	5	3	4
Скорость развёртывания	3	4	5	5	5	3
Технологическая гибкость	2	3	5	4	5	2
Устойчивость к сбоям	2	3	5	4	3	4
Сложность управления (1 – низкая, 5 – высокая)	2	3	5	3	4	3
Стоимость поддержки (1 – низкая, 5 – высокая)	2	3	5	4	4	3
Пригодность для больших команд	2	3	5	4	5	4

7 Обзор популярных фреймворков и платформ для WEB-разработки

Ниже приведены таблицы с наиболее популярными фреймворками для фронтенда и бэкенда, а также платформами для serverless-разработки.

Популярные фронтенд-фреймворки

Фреймворк	Язык	Ключевые особенности	Рекомендуемые сценарии
React	JavaScript / TypeScript	Высокая производительность, гибкость, огромная экосистема	Е-commerce, сложные пользовательские интерфейсы, SPA
Angular	TypeScript	Структурированный, enterprise-уровень, встроенные инструменты	Крупные структурированные приложения, корпоративные системы
Vue.js	JavaScript / TypeScript	Баланс простоты и функциональности	Проекты разного масштаба, где нужна гибкость
Next.js	TypeScript	SSR, SSG, полный стек	SEO-ориентированные приложения, full-stack проекты

Популярные бэкенд-фреймворки

Фреймворк	Язык		Ключевые особенности	Рекомендуемые сценарии
Spring Boot	Java		Надёжность, безопасность, интеграция с enterprise-системами	Банковские, телеком- и финтех-системы
Django	Python		Безопасность, масштабируемость, «батарейки в комплекте»	Крупные data-driven приложения, high-load ERP
ASP.NET Core	C#		Высокая производительность, интеграция с Windows	Enterprise-level ERP, системы на Microsoft Stack

Laravel	PHP		Простота, дружелюбная экосистема	Средние решения, ERP-rapid development
Express.js	JavaScript		Лёгкий, гибкий, для API и микросервисов	Микросервисы, REST/GraphQL API
NestJS	TypeScript		Модульная архитектура, enterprise-grade	Крупные бэкенд-системы на TypeScript

Платформы для serverless-разработки

Платформа	Провайдер	Ключевые особенности
AWS Lambda	Amazon	Самая зрелая платформа, широкая интеграция с сервисами AWS
Azure Functions	Microsoft	Интеграция с экосистемой Microsoft, поддержка многих языков
Google Cloud Functions	Google	Интеграция с GCP, простота использования

8 Варианты использования архитектур в компании

8.1 Стартапы и малые компании

Оптимален монолит с использованием современных фреймворков (Django, Laravel, Ruby on Rails). Позволяет быстро создать MVP, минимизировать затраты. По мере роста возможен переход к модульному монолиту.

8.2 Средние компании (50–200 сотрудников)

Рекомендуется многоуровневая архитектура с разделением на frontend, backend и БД. Стек: React/Angular (frontend), Django/ASP.NET Core (backend), SQL (PostgreSQL, MS SQL Server).

8.3 Крупные компании и высоконагруженные системы

Предпочтительна микросервисная архитектура для независимого масштабирования и быстрого вывода функций. Примеры: платформы

электронной коммерции, финансовые системы, социальные сети. Требуется «зрелая» DevOps-культура и автоматизированная инфраструктура.

8.4 Компании с переменной нагрузкой

Для сезонных или непредсказуемых нагрузок оптимальна serverless-архитектура – автоматическое масштабирование и оплата по факту.

8.5 Компании с несколькими фронтенд-командами

Внедрение микро-фронтендов позволяет каждой команде разрабатывать и развёртывать свои модули независимо.

8.6 Компании, работающие в российской юрисдикции с учётными задачами

Платформа 1С часто выступает как обязательное ядро для бухгалтерского и управленческого учёта. Её использование целесообразно в гибридной связке с микросервисами для специализированных функций.

9 Рекомендации по выбору архитектурного решения

На основе проведённого анализа можно сформулировать следующие рекомендации:

Проще всего начать с монолита. Для большинства проектов это самый быстрый и экономичный способ запуска продукта.

На микросервисы нужно переходить осознанно. Это оправдано только при достижении системой значительного размера, наличии нескольких независимых команд и зрелой DevOps-культуры.

Необходимо учитывать компетенции команды. Микросервисы и serverless требуют глубоких знаний распределённых систем, контейнеризации и мониторинга.

Необходимо оценивать совокупную стоимость владения. Учитывать не только разработку, но и сопровождение, инфраструктуру, обучение и миграцию.

Использовать гибридные подходы. Например, монолитное ядро для основной логики и вынос высоконагруженных функций в микросервисы или serverless-функции. В российской практике часто применяется гибрид «1С + микросервисы».

При выборе фреймворка необходимо ориентироваться на специфику проекта: для сложных интерфейсов – React/Vue, для enterprise-бэкенда – Spring Boot/ASP.NET Core, для учётных систем – 1С.

10 Заключение

Проведённый анализ показал, что рынок программного обеспечения для WEB-приложений представляет собой многослойную экосистему, включающую архитектурные стили (монолит, N-tier, микросервисы, serverless, композитные/headless), фреймворки разработки (как для фронтенда — React, Angular, Vue, так и для бэкенда — Django, Spring Boot, ASP.NET Core, Laravel), веб-серверы (IIS, Apache, Nginx) и специализированные платформы (Delphi 10.2, «1С:Предприятие»). Каждый из этих элементов имеет свои сильные стороны и ограничения, и их оптимальное сочетание зависит от контекста конкретного проекта.

Ключевые выводы по архитектурным стилям:

1. Монолит остаётся предпочтительным для стартапов и небольших проектов благодаря простоте разработки и низкой стоимости входа. Однако с ростом сложности он может стать препятствием для масштабирования.
2. Многоуровневая (N-tier) архитектура является золотой серединой для большинства корпоративных систем, обеспечивая чёткое разделение ответственности и умеренную гибкость.
3. Микросервисы – мощный, но сложный инструмент, оправданный только в крупных распределённых системах с высокой нагрузкой и наличием нескольких независимых команд. Переход на микросервисы должен быть осознанным и подкреплён зрелой DevOps-культурой.
4. Serverless идеально подходит для систем с переменной нагрузкой и событийно-ориентированных задач, но требует внимательного подхода к вопросам привязки к вендору и управления холодным стартом.
5. Композитные и headless архитектуры становятся стандартом для предприятий, стремящихся к гибкости, быстрой адаптации и снижению

зависимости от вендоров, позволяя заменять отдельные компоненты без полной перестройки системы.

По выбору фреймворков и платформ:

- Для создания сложных пользовательских интерфейсов лидируют React, Angular, Vue (с предпочтением React и Next.js для SEO-ориентированных проектов). Однако в 2025–2026 годах наблюдается тренд на Server-First подход, смещающий акцент с тяжёлых SPA в сторону серверного рендеринга и edge-вычислений.
- На бэкенде популярны Spring Boot (для надёжных enterprise-систем на Java), ASP.NET Core (для экосистемы Microsoft), Django (для быстрой разработки на Python) и Laravel (для PHP-проектов). Все они хорошо интегрируются с реляционными СУБД и поддерживают современные API-стандарты.
- Веб-серверы играют ключевую роль в развёртывании: IIS – оптимальный выбор для Windows-ориентированных организаций (глубокая интеграция с .NET и Active Directory), Nginx – лидер по производительности и масштабируемости, особенно в роли reverse-прокси, Apache – универсальное и надёжное решение с огромным сообществом.
- Специализированные платформы занимают свои ниши:
 - 1С:Предприятие является безальтернативным решением для бухгалтерского и управленческого учёта в России, обеспечивая высокую скорость разработки учётных систем и соответствие законодательству.
 - В корпоративных ландшафтах 1С часто выступает как ядро, интегрируемое через веб-сервисы с другими системами.
 - Delphi 10.2 Tokyo предоставляет возможности для создания высокопроизводительных серверных приложений и REST API, особенно востребованных в компаниях с существующим кодом на Delphi. Благодаря компиляции в нативный код и поддержке Linux, Delphi-сервисы могут эффективно работать в микросервисных и гибридных архитектурах.

Современный подход к проектированию информационных систем предполагает гибридные схемы, где монолитное ядро (например, 1С)

сочетается с микросервисами для высоконагруженных функций, а пользовательские интерфейсы строятся на SPA-фреймворках и раздаются через Nginx или IIS. Такой подход позволяет использовать сильные стороны каждого решения, минимизируя их недостатки.

Выбор архитектуры и технологического стека должен быть определен конкретными бизнес-требованиями, компетенциями команды и стратегией развития компании. Не существует универсального «лучшего» решения – важно уметь эволюционировать архитектуру по мере роста системы и изменения рыночных условий. В условиях активного внедрения AI, edge-вычислений и композитных подходов, ключевым фактором успеха становится способность команды быстро адаптироваться и выбирать наиболее подходящие инструменты для каждой задачи.

11 Список использованных источников

Söderman O. Comparison of web application architecture styles. Bachelor's Thesis, Aalto University, 2025.

Krzysztoń Ł., Łatwiński K., Plechawska-Wójcik M. Comparative analysis of selected aspects of web application architectures. Journal of Computer Sciences Institute, Vol. 34, Lublin University of Technology, 2025.

The problem of choosing a modern architectural solution for creating interactive applications and websites. Kuban State University of Technology, 2025.

Rashid R.S., Ismael A.A. The Role of Web Programming in Modern IT Solutions: Trends and Challenges. Journal of Information Systems Engineering and Management, 2025.

Client-Server Architecture for ERP Systems: A Comparative Analysis of Laravel, Django, and ASP.NET Core. IEEE Xplore, 2025.

Что такое микросервисная архитектура: плюсы и минусы. Agima.ru, 2025.

Web Application Architecture: A Comprehensive Exploration for 2025. Netguru, 2025.

Benefits and Challenges of Serverless Application Architecture. Google Cloud / Google Developer forums, 2025.

Serverless Architecture & Computing: Pros, Cons, Best Fits, and Solving Challenges. Splunk, 2025.

Architecture Styles. Azure Architecture Center, Microsoft Learn, 2025.

Evaluating Scalability and Modularity: A Comparative Study of Traditional Software Architectures. Zenodo, 2025.

Assessing the Impact of Asynchronous Communication on Resilience and Robustness: A Comparative Study of Microservice and Monolithic Architectures. Springer, 2025.

Официальная документация платформы «1С:Предприятие» (раздел «Архитектура», «Веб-клиент»), 2025.

1С:Предприятие 8.3. Руководство разработчика. – М.: 1С-Паблишинг, 2024.

Teti D. Delphi Cookbook – Third Edition. – Packt Publishing, 2018

Microsoft. Internet Information Services (IIS) Documentation. – URL: <https://learn.microsoft.com/en-us/iis/>

The Apache Software Foundation. Apache HTTP Server Documentation. – URL: <https://httpd.apache.org/docs/>

NGINX Documentation. – URL: <https://nginx.org/en/docs/>

Delphi 10.2 Tokyo Product Documentation. – URL: https://docwiki.embarcadero.com/RADStudio/Tokyo/en/Main_Page